

포인터에 관한 리포트

2024402035 장경민

목차

1. 기본 개념
2. 포인터와 배열
 1. 간단한 고찰
3. 포인터와 문자열
 1. 간단한 고찰
4. 포인터와 함수
5. 함수 포인터
6. 구조체 포인터
7. 이중 포인터
8. 참고 자료

1. 기본 개념

포인터는 메모리의 주소를 저장하는 자료형이다.

포인터는 다음과 같이 선언된다.

```
<type> *pt;
```

포인터는 선언될 때 어떤 타입의 포인터인지 알려줘야 한다.

```
int *pt, char *pt, double *pt...
```

메모리 속에 들어있는 어떠한 변수의 주소를 알고 싶으면 & 연산자를 사용한다.

```
int temp;  
&temp; // 예시) 1288634400
```

포인터(주소)에 저장된 값을 알고 싶으면 * 연산자를 사용한다.

```
int temp = 8;  
int *pt = &temp;
```

```
*pt; // *pt의 값은 8이다.
```

포인터는 다음과 같은 연산이 가능하다.

```
int *p1, *p2;
int i;
p1 = &i, p1 = p2 // 포인터에 대입
p1 + 1, p1 + i // 포인터와 정수 상수 or 포인터와 정수 변수의 덧셈.
                // 이때 p1의 바이트 크기 만큼 포인터가 이동한다.
p1 < p2 // 포인터 간의 비교
등등
```

포인터 간의 뺄셈은 약간 주의를 기울여야 한다.

```
int *p1, *p2;

p1 = (int *)malloc(sizeof(int) * 10);
p2 = p1 + 6;
p2 - p1; // 이것은 p1과 p2사이의 요소 수(offset)을 나타낸다. = 6
```

```
int *p1, *p2;

p1 = (int *)malloc(sizeof(int) * 10);
p2 = (int *)malloc(sizeof(int) * 50);
p2 - p1; // 이것은 아무런 의미가 없다. 동적으로 할당될 때 포인터의 위치가 달라지므로 실행할 때마다
다른 위치가 나올 수 있음
```

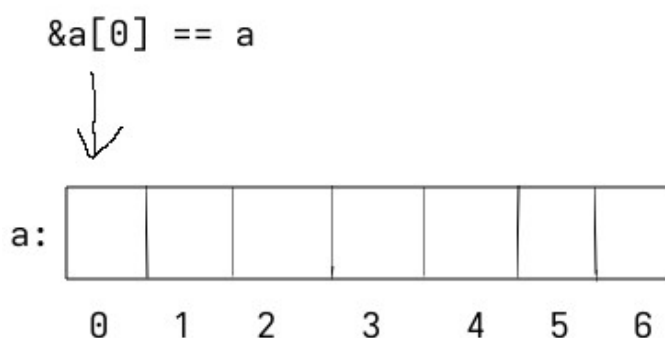
위와 같이 포인터의 뺄셈은 서로 연관 있는 포인터 만이 의미가 있는 값이 나온다.

포인터는 주로 다음과 같은 곳에서 사용된다.

1. 함수 내에서 원본 데이터 수정.
2. malloc과 같은 동적 메모리 할당
3. 1과도 연관이 있지만 함수에서 대용량 값(크기가 수만이 넘는 배열 등)의 복사 비용 낮추기

2. 포인터와 배열

포인터와 배열은 아주 많은 관련이 있다.



위와 같은 배열 a가 있을 때 a[0]의 주소인 &a[0]은 배열 이름 a와 같다.

포인터 변수에 배열의 시작 부 위치를 대입하면 포인터를 이용해 배열에 접근할 수 있다.

```
&a[0] == a

int a[7];
int *p;
p = a;
*(p + 3); // p[3]과 같은 값을 가진다.
```

배열로 할 수 있는 거의 대부분을 포인터도 할 수 있다.

하지만 몇몇 부분에서는 다르다.

```
int a[7], b[8];
int *p;

sizeof(a); // 28 // 컴파일러가 a의 크기를 할당해줌
sizeof(p); // 8

p = a;
p = b; // 가리키는 값 변경 가능
a = b; // 변경 불가능

void tem2(int a[10])
{
    sizeof(a); // 8 // 함수로 배열을 전달 받으면 포인터로 변환됨
}
```

따라서 배열 어느 부분에서는 포인터 상수 라고도 할 수 있다.

2.1. 간단한 고찰

포인터와 배열, sizeof등을 사용하면서 몇 가지 의문점이 들었다. 의문점과 그 것의 답을 찾아서 밑에 기록해 놓았다.

1. sizeof 함수

```
int list[8];
int *p;
int *q = (int *)malloc(sizeof(int) * 10);
sizeof(list); // 32
sizeof(p); // 8
sizeof(q); // 8
free(q);
```

Q) list의 크기는 32 나왔지만 p의 크기는 64비트 = 8로 나왔고 q는 sizeof(int) * 10인 40이 나올 것이라 예상하였지만 8이 나왔다. 그러면 배열과 포인터는 자신의 크기를 어디에 저장하고 있는가? 자신의 크기를 이용해

sizeof함수를 사용하는 게 아닌가? 자신의 크기가 저장되어 있지 않다면 free(q)를 하였을 때 어떻게 자신이 얼마나 할당되어 있는지 알고 해제한 것인가?

A) sizeof함수는 컴파일 타임 때 연산을 한 뒤 상수로 바꾼다. 따라서 동적으로 할당된 포인터의 크기는 알지 못한다. malloc함수로 동적 할당하면 리턴하는 포인터 앞에 크기 등을 저장해 놓는다. free함수는 이것을 보고 할당을 해제한다.

```
[헤더: 크기, 플래그 등] [실제 데이터 100바이트] // ClaudeAI에게 질문하여 확인
↑ p가 가리키는 곳
```

3. 포인터와 문자열

문자열은 char 형태인 배열이다. 따라서 앞의 설명을 거의 다 따라간다.

하지만 하나 특별한 점은 문자열의 끝은 NULL문자 혹은 '\0'으로 지정되어 있다는 것이다. 따라서 문자열 배열이나 문자열 포인터에서 이 문자열이 몇 글자인지 알지 못하여도 '\0'이 나오면 문자열이 끝났다는 것을 알 수 있다. 이를 이용해 printf("%s", text)등의 문자열 출력이나 문자열 다루기가 쉬워진다.

3.1. 간단한 고찰

포인터와 문자열



- 배열 형태로 선언된 매개변수에 1차원 배열의 이름이 나타내는 시작 주소값 전달

```
#include <stdio.h>

int main(void)
{
    char s1[] = "Hello World";
    char* s2 = "Hello World";

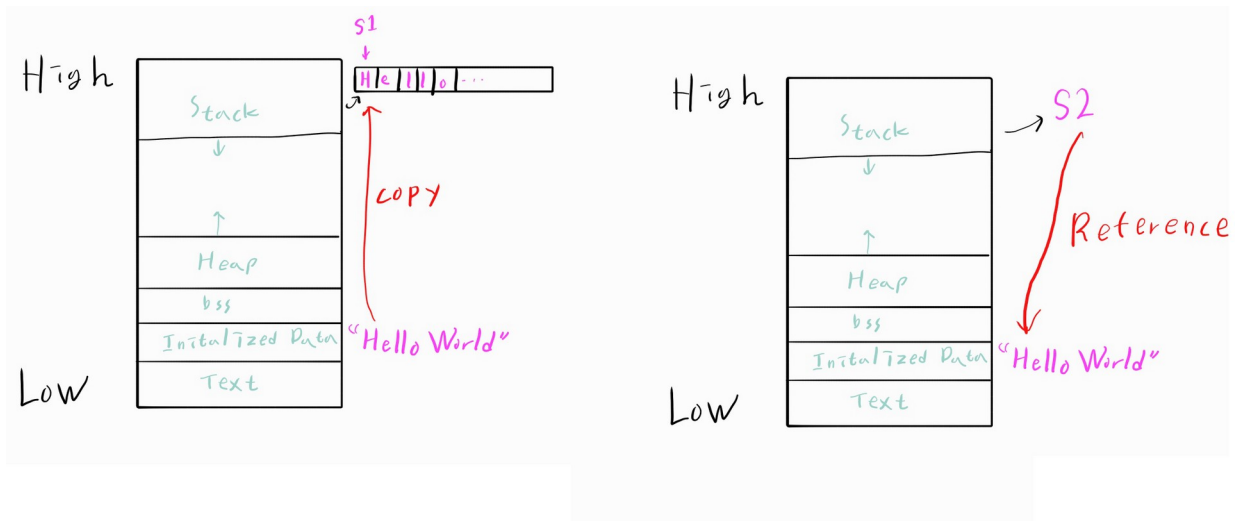
    s1[0] = 'h'; // 가능
    // s2[0] = 'h'; // 불가
```

s1의 경우 개별 요소값 수정 가능

s2의 경우 개별 요소값 수정 불가

Q) C언어 5일차 ppt 17쪽에 설명할 때 포인터인 s2의 경우 개별 요소 값이 수정 불가라 말하면서 마치 s2가 포인터여서 그렇다고 설명하였다. (잘못 들었을 수도 있다.) 위에서 배열과 포인터는 어떤 부분이 달라서 s1은 수정되고, s2는 수정되지 않는지 생각해 보았다.

A) 아마도 배열인 s1은 실제로 스택 영역에서 s1을 포인터로 가지고 크기가 "Hello World\0" = 12인 메모리를 할당하여 문자열을 복사한 후 저장해서 개별 요소값을 수정할 수 있다. 하지만 포인터인 s2는 DATA 영역에 있는 "Hello World\0" 참조하여 개별 요소 값을 수정 못하는 것 일 수 있다.



각각 $s1$ 과 $s2$ 를 메모리 레이아웃에 그려놓은 것이다.

4. 포인터와 함수

함수의 인수로 포인터를 전달하면 여러 가지 좋은 점이 있다.

1. 함수 외부의 원본 데이터를 수정할 수 있다.
2. 수 많은 값을 인수로 받을 때 복사 시간을 줄여 준다.
3. 동적 할당된 공간을 받기 위하여

등등의 장점이 있다.

```
void swap(int a, int b) // 교환 안됨
{
    int temp = a;
    a = b;
    b = temp;
}

void swap(int *a, int *b) // 교환 됨
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

typedef struct _S {int list[1000000];} S;

void def1(S s1) // S의 리스트 백 만개를 모두 복사한다.
{ /*...*/ };
```

```
void def2(S* s1) // S의 주소만 복사하여 자원을 아낄 수 있다.
{ /* ... */ };
```

5. 함수 포인터

함수의 포인터를 이용하여 여러가지 동작을 할 수 있다. 함수 포인터는 다른 포인터와 마찬가지로 배열이나 함수의 인수로 들어갈 수 있다.

```
int func1(int i1, int c1){ }
```



```
int (*func_p)(int, int); // 선언
```



```
func_p = func1; // 연결
func_p = func2; // 새로운 연결
```

위와 같이 선언하면 인수가 int와 int 이고, 리턴 값이 int인 func_p라는 이름의 함수 포인터를 생성한 것이다. 함수 포인터 또한 포인터 이므로, 다른 함수에 연결하거나, 배열, 구조체에 넣는 등 여러가지 일을 할 수 있다.

6. 구조체 포인터

구조체 포인터 또한 다른 타입의 포인터와 같다.

하지만 하나 다른 점이 있다면, 구조체 포인터의 요소에 접근할 때에는 -> 연산자를 사용할 수 있다. 구조체 요소에 접근할 때 쓰이는 . 연산자가 우선순위가 포인터의 * 보다 더 빨라서 괄호를 씌우면 접근할 수 있지만, 보기 불편해서 잘 쓰이지 않는다.

```
typedef struct _S {int a; char b;} S;
S *pS;
```



```
S tamp;
pS = &tamp;
```



```
pS->a; // 구조체 포인터에 접근할 때에는 -> 연산자를 사용 할 수 있다.
*pS.a // 연산자 우선순위에 따라 .연산이 먼저 나온다.
(*pS).a; // 괄호를 씌우면 해결되긴 한다. 하지만 보기에 안 좋다.
```

7. 이중 포인터

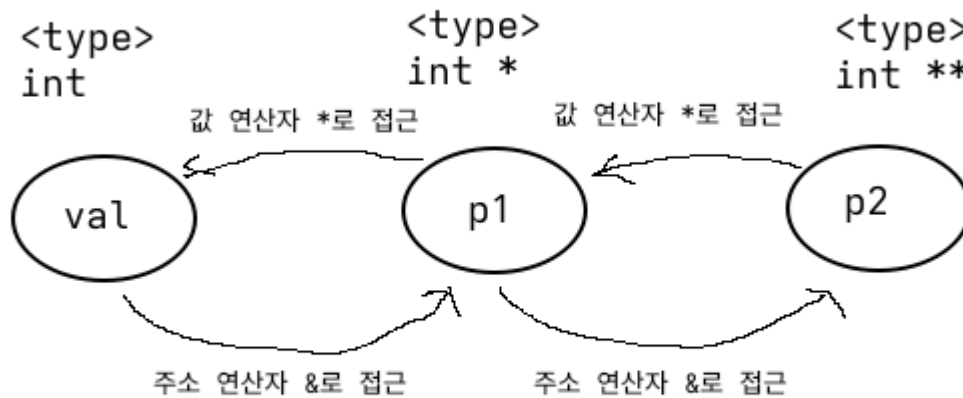
이중 포인터는 포인터를 가리키는 포인터이다.

```
int val = 5;
```

```
int *p1;
int **p2;

p1 = &val; // p1에는 val의 주소가 저장되어 있다.
p2 = &p1; // p2에는 p1의 주소가 저장되어 있다.

val == *p1 == **p2
```



위와 같이 **로 선언하고, 주소 연산자&, 값 연산자*로 접근할 수 있다.

이중 포인터는 다음과 같은 곳에서 사용된다.

1. 함수에 포인터의 주소를 전달해 주어야 할 때.
2. 2차원 배열을 동적으로 만들 때.
3. 2번과 비슷한 예시인 문자열 배열을 만들 때.

```
char **list = (char **)malloc(sizeof(char *) * 3);
*(list) = "text 00";
*(list + 1) = "text 01";
*(list + 2) = "text 02";

char *temp = *(list + 1);
*(list + 1) = *(list + 0);
*(list + 0) = temp; // list = {"text 01", "text 00", "text 02"}
```

위와 같이 문자열 배열을 사용하거나 이차원 배열을 동적으로 만들 때 이중 포인터를 사용한다. 필요에 따라서 삼중 사중 포인터 또한 만들 수 있다.

8. 참고 자료

강의 자료, (Kernighan의) C 언어 프로그래밍 2판, 컴퓨터 구조 및 설계 RISC-V 2판, ClaudeAI.